

LAPORAN
INFRASTRUKTUR DAN PLATFORM BIG DATA

**Sistem Prediksi Kualitas Udara (*Air Quality Index*) Kota
Surakarta**

Dosen Pengampu:
Fajar Muslim S.T., M.T.



Disusun Oleh:
Trisha Garnis Wahningyun (L0224012)
Kayla Maharani Muzaki (L0224050)

PROGRAM STUDI S1 SAINS DATA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET
2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
PENDAHULUAN.....	3
1.1. Latar Belakang.....	3
1.2. Tujuan.....	3
1.3. Ruang Lingkup.....	3
BAB II	
BATCH PROCESSING.....	5
2.1. Konsep dan Arsitektur Batch Processing.....	5
2.2. Alur Kerja dan Implementasi Pipeline.....	5
BAB III	
STREAM PROCESSING.....	6
3.1. Konsep dan Karakteristik Real-time Processing.....	6
3.2. Perbandingan dengan Batch Processing.....	6
BAB IV	
INTEGRASI DATA PIPELINE DENGAN ML.....	7
4.1. Proses Integrasi Pipeline.....	7
4.2. Training dan Inference Model ML.....	7
BAB V	
MONITORING DAN LOGGING.....	8
5.1. Mekanisme Monitoring Sistem.....	8
5.2. Mekanisme Logging dan Observability.....	8
BAB VI	
KEAMANAN DAN GOVERNANCE BIG DATA ON-PREMISE.....	9
6.1. Keamanan Sistem On-Premise.....	9
6.2. Data Governance.....	9
[Deskripsikan tata kelola data dalam sistem, meliputi: Data Quality, Metadata management, mekanisme Audit trail, dan Compliance.].....	9
BAB VII	
KESIMPULAN DAN EVALUASI.....	10
7.1. Kesimpulan.....	10
[Rangkuman hasil implementasi platform Big Data end-to-end, apakah sudah memenuhi arsitektur yang direncanakan.].....	10
7.2. Evaluasi Performa Sistem.....	10
[Dokumentasikan hasil pengujian dan evaluasi performa sistem yang telah berjalan pada lingkungan VM lokal serta troubleshooting yang telah dilakukan.].....	10
LAMPIRAN.....	11

BAB I

PENDAHULUAN

1.1. Latar Belakang

Kebutuhan memantau kualitas udara di kota-kota besar makin mendesak, tapi kebanyakan sistem yang ada sekarang bergantung pada AWS, GCP, atau Azure artinya ada biaya langganan dan harus punya koneksi internet yang stabil terus. *AQI Watch* Surakarta dibangun sebagai *pipeline big data end-to-end* yang jalan *on-premise* di satu VM lokal, diatur memakai Docker Compose. Total ada 15 service dalam satu node yaitu Kafka untuk ingestion, Spark untuk processing, PostgreSQL dan MinIO untuk storage, Airflow untuk scheduling, MLflow untuk *machine learning*, dan Prometheus/Grafana untuk monitoring. Semuanya berjalan dalam satu node tanpa ketergantungan *cloud provider*. Pendekatan ini memastikan portabilitas, biaya operasional minimal, dan kemandirian infrastruktur.

1.2. Tujuan

Berdasarkan Capaian Pembelajaran Mata Kuliah (CPMK), proyek ini bertujuan untuk:

- a. Mengimplementasikan platform *Big Data* mencakup *storage*, *processing*, dan *streaming* pada lingkungan VM.
- b. Mengelola *container* dan orkestrasi platform data secara lokal.
- c. Membangun *pipeline Big Data end-to-end* serta melakukan *monitoring*, *troubleshooting*, dan evaluasi performa sistem.
- d. Menyusun dokumentasi teknis dan mempresentasikan solusi infrastruktur Big Data.

1.3. Ruang Lingkup

Proyek ini dibatasi pada cakupan berikut, mulai dari lokasi pemantauan sampai dokumentasi yang dihasilkan:

- a. Lokasi mencakup 5 kecamatan di Kota Surakarta (Banjarsari, Jebres, Laweyan, Pasar Kliwon, Serengan).
- b. Data yang diambil meliputi 6 parameter polutan (PM2.5, PM10, CO, NO2, SO2, O3), ISPU, dan 5 parameter meteorologi (suhu, kelembaban, kecepatan angin, curah hujan, tutupan awan) dari Open-Meteo API.
- c. Pipeline menjalankan *batch processing* untuk data historis 1 tahun, serta *streaming real-time* dengan interval 60 detik.
- d. Infrastruktur terdiri dari 15 *container on-premise* dalam 1 node VM, diorkestrasi dengan Docker Compose.

- e. *Processing* dilakukan dengan Apache Spark, dipakai untuk batch ETL sekaligus *streaming analytics*.
- f. *Storage* menggunakan PostgreSQL untuk data relasional dan MinIO untuk object storage yang kompatibel S3.
- g. *Machine Learning* memanfaatkan *Random Forest Classifier* untuk prediksi kategori ISPU, ditambah K-Means untuk clustering stasiun, dengan MLflow sebagai *model registry*.
- h. Scheduling diatur lewat Apache Airflow, dengan DAG harian (*batch_ingest*) dan mingguan (*ml_retrain*).
- i. Monitoring dilakukan dengan Prometheus dan Grafana untuk dashboard, serta notifikasi Telegram lewat Alertmanager.
- j. Dokumentasi mencakup metadata database, tutorial *demo end-to-end*, panduan dashboard, dan *troubleshooting*.

1.4. Batasan Proyek

Di luar cakupan di atas, ada beberapa batasan teknis yang perlu disadari sejak awal, terutama menyangkut akurasi data dan keandalan infrastruktur:

- a. Data bersumber dari Open-Meteo API (hasil model, bukan sensor fisik).
- b. Cakupan lokasi hanya 5 titik koordinat kecamatan di Surakarta.
- c. Seluruh infrastruktur jalan di 1 node VM, tanpa high-availability atau clustering.
- d. Kafka pakai single-broker, tanpa replikasi.
- e. Perhitungan ISPU mengacu ke Permen LHK No. 14/2020, tanpa parameter HC.
- f. Model Random Forest masih versi baseline, belum lewat hyperparameter tuning lanjutan.
- g. Tidak ada autentikasi antar service internal (*trusted network*).
- h. Retensi data mentah 90 hari tanpa archival otomatis.

BAB II

BATCH PROCESSING

2.1. Konsep dan Arsitektur Batch Processing

Batch processing adalah metode pemrosesan data yang dilakukan dalam satu atau beberapa kumpulan (*batch*) secara periodik, bukan secara *real-time*. Dalam pendekatan ini, data terkumpul terlebih dahulu selama periode waktu tertentu, baru kemudian diproses secara bersamaan dalam satu kali eksekusi. Metode ini cocok untuk volume data besar yang tidak membutuhkan latensi rendah, namun membutuhkan akurasi dan ketelitian hasil yang tinggi.

Pada proyek ini, *batch processing* diterapkan melalui dua tahap utama. Tahap pertama adalah *extract*, yaitu pengambilan data historis 1 tahun dari Open-Meteo API melalui script `batch_extract.py`. Tahap kedua adalah transform dan load, yaitu pembersihan data, penghitungan ISPU, dan agregasi harian dilakukan oleh Spark melalui script `batch_etl.py`. Hasilnya disimpan ke PostgreSQL (tabel `daily_aqi`) sebagai *source of truth*, serta ke MinIO (bucket `processed/`) dalam format Parquet.

Arsitektur *batch processing* pada proyek ini dirancang berbasis *on-premise* dengan berbagai komponen yang saling terintegrasi:

- a. Apache Spark digunakan sebagai mesin pemrosesan utama karena kemampuannya dalam menangani data skala besar secara distributif. Spark membaca file CSV dari MinIO, melakukan transformasi, dan menulis hasil ke PostgreSQL maupun MinIO.
- b. MinIO berfungsi sebagai object storage S3-compatible untuk menyimpan data mentah (CSV) dan data hasil pemrosesan (Parquet).
- c. PostgreSQL sebagai database relasional utama yang menyimpan data harian hasil agregasi.
- d. Apache Airflow sebagai *orchestrator* yang menjadwalkan eksekusi *pipeline batch* secara otomatis.

Seluruh komponen diorkestrasi melalui Docker Compose dengan 15 service dalam satu node VM.

2.2. Alur Kerja dan Implementasi Pipeline

Alur kerja batch pipeline pada proyek ini mengikuti pola Extract-Transform-Load (ETL) yang terdiri dari tiga tahap berurutan.

- a. Extract (Ekstraksi Data)

Script `batch_extract.py` dijalankan untuk mengambil data historis dari Open-Meteo API selama 365 hari ke belakang. Proses ini mengambil dua jenis data secara terpisah: data

kualitas udara (PM2.5, PM10, CO, NO2, SO2, O3) dari Air Quality API, dan data meteorologi (suhu, kelembaban, angin, curah hujan, tutupan awan) dari Historical Weather API. Setiap data kemudian dikombinasikan berdasarkan stasiun, dilengkapi dengan perhitungan ISPU, dan diunggah ke MinIO bucket raw/ dalam format CSV dengan penamaan unik berbasis timestamp.

b. Transform (Pembersihan dan Agregasi)

Script `batch_etl.py` dijalankan melalui Apache Spark. Proses dimulai dengan mengunduh file-file CSV dari bucket raw/ MinIO, lalu melakukan pembersihan data (filter null, validasi range nilai, konversi tipe data), penghitungan kategori ISPU menggunakan UDF berdasarkan Permen LHK No. 14/2020, dan agregasi data per stasiun per hari. Hasil agregasi mencakup rata-rata polutan, total curah hujan, dan jumlah record per hari per stasiun.

c. Load (Penyimpanan Hasil)

Data hasil transformasi dimuat ke dua destinasi secara bersamaan: PostgreSQL tabel `daily_aqi` melalui JDBC dengan mekanisme upsert (ON CONFLICT DO NOTHING), serta ke MinIO bucket `processed/` dalam format Parquet yang dipartisi berdasarkan `station_id` dan `date`.

d. Penjadwalan (Scheduling)

Pipeline batch diorkestrasi oleh Apache Airflow melalui DAG `batch_ingest` yang dijadwalkan berjalan setiap hari pada pukul 06:30 WIB. DAG ini memanfaatkan operator `PythonOperator` untuk menjalankan `batch_extract.py`, diikuti oleh `SparkSubmitOperator` untuk menjalankan `batch_etl.py`. Setiap eksekusi dilakukan pencatatan audit ke tabel `pipeline_audit` untuk memastikan *traceability*.

BAB III

STREAM PROCESSING

3.1. Konsep dan Karakteristik *Real-time Processing*

[Jelaskan konsep stream processing dan bagaimana sistem menangani aliran data secara real-time. Jelaskan latensi, throughput, dan fault tolerance.]

3.2. Perbandingan dengan *Batch Processing*

[Sediakan perbandingan analisis mengapa proyek ini membutuhkan stream processing selain batch processing dan kapan masing-masing digunakan.]

BAB IV

INTEGRASI DATA PIPELINE DENGAN ML

4.1. Proses Integrasi Pipeline

[Jelaskan bagaimana data yang telah diproses dalam pipeline diumpankan ke dalam model Machine Learning.]

4.2. *Training* dan *Inference* Model ML

[Uraikan tahapan untuk proses training (misalnya jadwal retraining model) dan mekanisme inference atau prediksi model dalam ekosistem on-premise.]

BAB V

MONITORING DAN LOGGING

5.1. Mekanisme Monitoring Sistem

[Jelaskan tools monitoring yang digunakan untuk memantau metrics sistem seperti penggunaan resource komputasi dan performa infrastruktur data.]

5.2. Mekanisme *Logging* dan *Observability*

[Jelaskan strategi pengumpulan log untuk troubleshooting sistem, serta bagaimana observabilitas secara keseluruhan dikelola untuk memastikan ketersediaan layanan pipeline end-to-end.]

BAB VI

KEAMANAN DAN *GOVERNANCE BIG DATA ON-PREMISE*

6.1. Keamanan Sistem *On-Premise*

[Deskripsikan penerapan keamanan yang meliputi: Authentication (Otentikasi), Authorization (Otorisasi pengguna), Network security (Keamanan jaringan), dan Data protection (Perlindungan data).]

6.2. *Data Governance*

[Deskripsikan tata kelola data dalam sistem, meliputi: Data Quality, Metadata management, mekanisme Audit trail, dan Compliance.]

BAB VII

KESIMPULAN DAN EVALUASI

7.1. Kesimpulan

[Rangkuman hasil implementasi platform Big Data end-to-end, apakah sudah memenuhi arsitektur yang direncanakan.]

7.2. Evaluasi Performa Sistem

[Dokumentasikan hasil pengujian dan evaluasi performa sistem yang telah berjalan pada lingkungan VM lokal serta troubleshooting yang telah dilakukan.]

LAMPIRAN